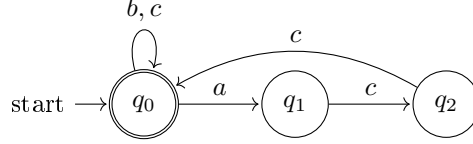


QUESTION 1

(a). Let $L = \{w \in \{a, b, c\}^* : \text{each occurrence of } a \text{ in } w \text{ is immediately followed by } cc\}$. Draw a minimal DFA for L .

Solution: Here is a minimal DFA for L :



(b). Let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ be a finite, deterministic DFA. Let

$L = \{w \in \Sigma^* : \mathcal{A} \text{ accepts } w, \text{ and the run of } \mathcal{A} \text{ on } w \text{ visits each state at least once}\}$

We claim that $L \in REG$.

Solution: Let $q \in Q$, and define

$L_q = \{w \in \Sigma^* : \mathcal{A} \text{ accepts } w, \text{ and the run of } \mathcal{A} \text{ on } w \text{ visits } q \text{ at least once}\}$

We claim that $L_q \in REG$. Indeed, consider the following DFA, $\mathcal{A}_q = \langle Q \times \{0, 1\}, \Sigma, \delta_q, (q_0, 0), F \times \{1\} \rangle$, where δ_q is defined as:

$$\delta_q((s, k), \sigma) = \begin{cases} (\delta(\sigma, s), k) & : \delta(\sigma, s) \neq q \\ (\delta(\sigma, s), 1) & : \delta(\sigma, s) = q \end{cases}$$

That is, $\mathcal{A}_q$ “remembers” whether it has visited q , and accepts iff $\mathcal{A}$ accepts and q has been visited.

Let $w \in L_q$. During the run of $\mathcal{A}_q$ on w , if we consider only the left hand side of $Q \times \{0, 1\}$, the run is identical to the run of $\mathcal{A}$ on w . Hence, as $w \in L_q$, this run visited q , and thus, on the transition leading to this visit, we had the transition to $(q, 1)$. Consequently (By induction on $|w|$), all states from this point on are of the form $(s, 1)$, as is the final state $s \in F$, the run ends on $F \times \{1\}$, and $w \in L(\mathcal{A}_q)$.

If $w \in L(\mathcal{A})$, an identical argument gives us that the run of $\mathcal{A}$ on w ends in F , and that (Again, by induction on $|w|$), we must have visited q . Thus, $L(\mathcal{A}_q) = L_q$, and $L_q \in REG$.

Now, note that

$$L = \bigcap_{q \in Q} L_q$$

and as Q is finite, this is a finite intersection. We have shown in class that REG is closed under finite intersection, therefore $L \in REG$.

(c). Let $L \subseteq \{0, 1\}^*$ such that $\{0^n 1^n : n \in \mathbb{N}\} \subseteq L$, and for every $w \in L$, it holds that $\#_0(w) = \#_1(w)$. We claim that $L \notin REG$.

Solution: Assume towards a contradiction that such an L is regular. Let $p > 0$ be the pumping constant for L , and consider $w = 0^p 1^p$. It holds that $w \in L$, and that $|w| = 2p > p$. Hence, from the Pumping Lemma, there exist some $x, y, z \in \{0, 1\}^*$ such that:

- (1) $w = xyz$
- (2) $|y| > 0$
- (3) $|xy| \leq p$
- (4) For every $i \in \mathbb{N}$, $xy^i z \in L$

Note that this implies that $y = 0^n$ for some $0 < n \leq p$. However, consider the word $w' = xy^0 z = 0^{n-p} 1^p$. By the above, $w' \in L$, while $\#_0(w') = n - p < n = \#_1(w')$, in contradiction to L 's definition.

Hence, the Pumping Lemma cannot hold for L , and consequently, L is not regular.

QUESTION 2

(a). Let $L_1 \in P$ and $L_2 \in R$ s.t. $L_1, L_2 \neq \emptyset, \Sigma^*$. We claim that $L_1 \leq_p L_2$.

Solution: Let $w_\perp \in L_2$ and $w_\top \notin L_2$ (Such words exist, as $L_2 \neq \emptyset, \Sigma^*$). We define a reduction f from L_1 to L_2 as follows. On input $w \in \Sigma^*$:

- (1) Check whether $w \in L_1$.
- (2) If $w \in L_1$, return $w_\perp$.
- (3) Otherwise, return $w_\top$.

Correctness: Given $w \in L_1$, we return $f(w) = w_\perp \in L_2$. Given $w \notin L_1$, we return $f(w) = w_\top \notin L_2$, as required.

Additionally, note that as $L_1 \in P$, step 1 is computable.

Run-time: Again, both $w_\perp$ and $w_\top$ are constant, and as $L_1 \in P$, we can check whether $w \in L_1$ in time polynomial in $|w|$. Thus, all three steps are polynomial in $|w|$, as required.

(b). Let

$$L_{1-2} = \{ \langle M_1, M_2 \rangle : M_1 \text{ halts on at least one input of odd length,} \\ \text{and } M_2 \text{ halts on at least one input of even length} \}$$

We claim that $L_{1-2} \in \text{RE}$.

Solution: We define a *TM* that recognizes L_{1-2} as follows. On input $\langle M_1, M_2 \rangle$, using a counter $n \in \mathbb{N}$, on the n -th iteration:

- (1) Run M_1 on all odd-length words of length $\leq n$ for n steps. If at any point M_1 halts, set some flag as “true”.
- (2) Run M_2 on all even-length words of length $\leq n$ for n steps. If at any point M_2 halts, set some other flag as “true”.
- (3) If at the end of the n -th iteration both flags are true, accept (note that we never reset either flag to false).

Correctness: If M_1 halts on some odd-length word w_1 , it does so within n_1 steps, for some $n_1 \in \mathbb{N}$. Consequently, on the $m_1 = \max\{|w_1|, n_1\}$ -th iteration, our TM will set the first flag as true. Similarly, if M_2 halts on some even-length word w_2 , it does so within n_2 steps, for some $n_2 \in \mathbb{N}$, and thus, on the $m_2 = \max\{|w_2|, n_2\}$ -th iteration, our TM will set the second flag as true. Hence, if $\langle M_1, M_2 \rangle \in L_{1-2}$, on the $n = \max\{m_1, m_2\}$ -th iteration, both flags will be set as true, and our TM will correctly accept.

In the converse, if $\langle M_1, M_2 \rangle \notin L_{1-2}$, assume w.l.o.g. that M_1 never halts on any odd-length word. Our TM will then never set the first flag as true (As we only ever simulate M_1 on odd-length words), and consequently, will never accept. The case in which M_2 never halts on even-length words is similar.

Computability: We have seen that simulating a TM for a set number of steps is possible, as is flag-setting and -checking.

(c). Let L_{1-2} be as above, we claim that $L_{1-2} \notin \text{coRE}$

Solution: We show the claim by reducing $HALT_{TM} \leq_m L_{1-2}$. As we know that $HALT_{TM} \notin \text{coRE}$, the Reduction Theorem then implies that $L_{1-2} \notin \text{coRE}$. Our reduction works as follows. Set some $\sigma \in \Sigma$. Given $\langle M, w \rangle$, return $\langle D, D \rangle$, where D is a TM that ignores its input, and returns the result of running M on w .

Correctness: If $\langle M, w \rangle \in HALT_{TM}$, then D halts on any $x \in \Sigma^*$, in particular, as Σ is not empty, it halts on some word of even length, and on some word of odd length, and thus, $\langle D, D \rangle \in L_{1-2}$.

If $\langle M, w \rangle \notin HALT_{TM}$, then D does not halt on any $x \in \Sigma^*$, and in particular, it never halts on any word of even length, and thus, $\langle D, D \rangle \notin L_{1-2}$.

Computability: We have seen (Repeatedly) that a TM such as D is computable.

QUESTION 3

(a). There cannot exist a deterministic TM M , with $\Sigma = \{0, 1\}$, tape-size bounded by $3n^2$, $|\Gamma| = 5$, $|Q| = 10$, such that there exists some input x of length 3 on which M halts in exactly 10^{30} steps.

Solution: The number of configurations that a TM with space complexity $s(n)$ is bounded by $|Q| \cdot |\Gamma|^{s(n)} \cdot s(n)$. Indeed, a configuration consists of a state in Q , the content of the tape (where we need to choose a letter in Γ for each of the $s(n)$ cells, and a location for the reading head. In our case, $s(n) = 3n^2$ and $n = 3$. So the number of configurations is $10 \cdot 5^{27} \cdot 27$, which is strictly smaller than 10^{30} . Hence, a run of the TM of the input word that has 10^{30} steps would repeat a configuration, and would loop forever, without halting.

(b). An instance of the TSPT is of the form $\langle G, W, S \rangle$, where

- (1) $G = \langle V, E \rangle$ is a complete, undirected graph
- (2) $W : E \rightarrow \mathbb{N}$, s.t. for every $x, y, z \in V$ it holds that $W((x, y)) \leq W((x, z)) + W((z, y))$

Such an instance is in the language TSPT if G admits a Hamiltonian circuit of weight no more than S .

We claim that $TSPT \in \text{NPC}$.

Solution: First, $TSPT \in NP$, as we have seen that given an instance $\langle G, W, S \rangle$, and a possible Hamiltonian circuit of weight no more than S :

- (1) Checking that the graph G is complete is polynomial in $|V|$ (As there are no more than $|V|^2$ edges).
- (2) Checking that W is positive, and respects the triangle inequality is polynomial in $|V|$ (As there are no more than $(|V|^2)^3$ triplets of edges).
- (3) Checking that the path is a legal Hamiltonian circuit is polynomial in $|V|$.
- (4) Checking that the weight of the path is no more than S is polynomial in $|V|$ and the representation of W and S .

Recall that $U - HAMCYCLE \in NPC$. Thus, from the Reduction Theorem, it is enough to show that $U - HAMCYCLE \leq_p TSPT$. We define a reduction as follows. Given a graph $G = \langle V, E \rangle$, we return $\langle G', W, S \rangle$, where:

- (1) $G' = \langle V, V \times V \rangle$
- (2) For every $(x, y) \in V \times V$,

$$W((x, y)) = \begin{cases} 0 & : x = y \\ 1 & : (x, y) \in E \\ 2 & : otherwise \end{cases}$$

- (3) $S = |V|$

Note that for every $x, y, z \in V$, it holds that $W((x, y)) \leq W((x, z)) + W((z, y))$, as:

- If $x = y$, the lhs is 0.
- If $x = z$ or $y = z$, the equality is immediate.
- Otherwise, $W((x, y))$ is at most 2, while both $W((x, z))$ and $W((z, y))$ are at least 1.

Hence, $\langle G', W, S \rangle$ is an instance of $TSPT$.

Correctness: If $G \in U - HAMCYCLE$, then it admits a Hamiltonian circuit. This circuit also exists in G' , and its weight is exactly $|V| = S$ (As all of its edges are also edges in G). Thus, $\langle G', W, S \rangle \in TSPT$.

If $\langle G', W, S \rangle \in TSPT$, then G' admits a Hamiltonian circuit. This circuit must have exactly $|V|$ edges. However, as none of them can be self-edges, all of them must have weight of exactly 1. Thus, all of these edges must also exist in G , and consequently, $G \in U - HAMCYCLE$.

Computability: Defining G' , W , and S are all steps that we have seen to be possible in class, as we have seen methods of graph, function, and number encoding.

Run-time: Defining G' from G requires at most $|V|^2$ new edges, which is polynomial in $|V|$. Similarly, defining W requires defining $|V|^2$ cases, each of constant size. Finally, defining $S = |V|$ is polynomial in $|V|$.

(c). Let *MoreSAT* be the language of 3 - CNFs who admit more satisfying assignments than non-satisfying ones. We claim that *MoreSAT* $\in$ PSPACE.

Solution: We prove that MoreSAT is in PSPACE.

Consider the following TM M that decides MoreSAT : Given a 3CNF formula φ with n variables, M goes over all the 2^n assignments to the variables and increases a binary counter for each one that satisfies φ . Afterwards, M checks whether the counter is strictly larger than 2^{n-1} , and accepts if this is true.

Clearly, M accepts φ only if it has more satisfying than un-satisfying assignments. As for the space complexity, the maintenance of the counter requires $O(n)$ bits, so does its comparison with 2^{n-1} , and so does going over all assignments. For example, as we saw in class when we described a PSPACE algorithms for SAT or VAL, we can start with the assignment that assigns F to all variables and go over all assignments in a lexicographic order. Finally, the evaluation of φ for each assignment requires polynomial space (which is reused for all assignments).